Uso da I.A Codex, GPT-5.5, Altíssimo. 3 Janelas de contexto utilizadas no total. Expiração dos tokens no dia da apresentação. Os prompts de dúvidas em relação a nomenclaturas e exemplos foram ignorados.

Prompt 1: É possível aplicar algoritmos genéticos na bolsa de valores, que analise a alta e baixa da bolsa/açao, e preveja o próximo preço dela para fazer a melhor compra/venda?

Resposta resumida:

Um algoritmo genético pode otimizar, por exemplo:

- janelas de médias móveis;
- limites de RSI/MACD/volatilidade;
- regras de stop loss e take profit;
- combinação de indicadores;
- parâmetros de uma rede neural ou outro modelo;
- tamanho da posição e gestão de risco.

O ponto crítico é a validação. Em séries temporais, não se deve embaralhar os dados como em problemas comuns de machine learning; o teste precisa acontecer sempre "no futuro" em relação ao treino. A própria documentação do `TimeSeriesSplit` do scikit-learn destaca esse tipo de separação temporal para evitar treinar com informação futura e testar no passado: scikit-learn TimeSeriesSplit.

Também precisa tomar cuidado com:

- **overfitting**: o algoritmo encontra uma regra perfeita para o passado, mas ruim no futuro;
- **custos reais**: corretagem, spread, slippage, impostos, taxas;
- **liquidez**: algumas ações parecem lucrativas no backtest, mas não executam bem no mercado real;
- **risco de day trade**: operações muito curtas são altamente arriscadas; a SEC/Investor.gov alerta que day trading pode gerar perdas substanciais rapidamente: Investor.gov;
- **restrições operacionais**: dependendo do país/corretora, há regras específicas para day trade e margem; a FINRA, por exemplo, descreve exigências para pattern day traders nos EUA: FINRA.

Prompt 2: Uso de artigos que relatam uma forma de utilizar AG em Ações da Bolsa.

PREVISÃO DE AÇÕES COM MO... | Algoritmos genéticos na otimiza...

O que esses 2 artigos relatam sobre utilizar algoritmos genéticos na bolsa? Qual a abordagem que eles utilizam?

quinta-feira, 16:22

Artigo	Uso do algoritmo genético	Objetivo
LSTM + AG	Otimizar parâmetros de uma rede neural LSTM	Prever o próximo preço da ação
Otimização de carteiras	Otimizar os pesos de uma carteira	Melhorar relação risco-retorno

Prompt 3: Gostaria de fazer esse algoritmo genético utilizando a ação do ouro. Isso é viável?

```text
GC=F histórico
-> calcular indicadores: médias móveis, RSI, volatilidade, retornos
-> algoritmo genético cria várias estratégias
-> cada estratégia é testada no passado
-> fitness mede lucro, acerto, drawdown e risco
-> melhores estratégias sobrevivem
```

A função de avaliação poderia ser:

```text
fitness = retorno_total
          - penalidade_por_drawdown
          - penalidade_por_excesso_de_operacoes
          - custos_estimados
```

O alvo pode ser definido de duas formas:

```text
Regressão:
prever o preço de fechamento de amanhã

Classificação:
prever se amanhã sobe ou cai
```

Para operação, a classificação costuma ser mais prática:

```text
se probabilidade de alta > limite:
    comprar

se probabilidade de baixa > limite:
    vender ou ficar fora
```

Prompt 4: Eu teria um resultado promissor utilizando apenas AG?

Sim, você pode ter um resultado promissor usando **apenas algoritmo genético**, sem LSTM. Na verdade, para começar, eu acho até melhor: fica mais simples, mais interpretável e mais fácil de testar.

Mas tem um ponto crucial: o AG pode "ficar bom demais no passado" e ruim no futuro. Isso se chama **overfitting de backtest**. Como o AG testa muitas combinações de regras, ele pode encontrar uma estratégia que encaixa perfeitamente nos dados antigos por acaso. Esse risco é bem conhecido em finanças; Bailey, Borwein, López de Prado e Zhu discutem justamente a "probabilidade de overfitting em backtests" em estratégias de investimento: SSRN.

A abordagem com AG puro seria assim:

```text
dados históricos do ouro
-> criar indicadores
-> gerar várias estratégias aleatórias
-> testar cada estratégia no passado
-> selecionar as melhores
-> cruzar e mutar os genes
-> repetir por várias gerações
-> testar a melhor estratégia em dados que ela nunca viu
```

Esse indivíduo vira uma regra:

```text
comprar quando média curta > média longa e RSI < 35
vender quando RSI > 70, stopar em -2% ou realizar em +4%
```

A parte mais importante é **não deixar o algoritmo treinar até os dias atuais e depois dizer que ele acertou o passado**. O ideal é separar:

```text
treino: o AG evolui as estratégias
validação: você escolhe ajustes
teste: você mede o resultado final sem mexer mais
```

Ou melhor ainda: usar **walk-forward**:

```text
2010-2015 -> treina
2016 -> testa

2011-2016 -> treina
2017 -> testa

2012-2017 -> treina
2018 -> testa
...
```

Prompt 5: Entao vamos começar a parte de programação. Fazendo o 1. backtest no nosso código.

Rodei um backtest real no `GC=F`, de `2020-01-02` até `2024-12-30`, com uma estratégia fixa de médias móveis + RSI:

```text
retorno_total: 19,63%
retorno_anual: 3,66%
buy_and_hold: 70,95%
max_drawdown: 9,53%
trades: 34
win_rate: 50,00%
profit_factor: 1,65
```

Isso é uma boa primeira leitura: a estratégia deu lucro, mas ficou bem abaixo do buy and hold no período. Esse é exatamente o tipo de informação que o backtest precisa revelar antes de otimizarmos com AG.

Prompt 6: Agora faça o walk-forward backtest.

Também atualizei o **README.md** com o comando.

Rodei um teste real curto com `GC=F`, usando `2020-01-01` até `2024-12-31`, treino de 756 candles e teste de 126 candles. Resultado fora da amostra:

```text
capital_final: 9910.52
retorno_total: -0.89%
retorno_anual: -0.60%
buy_and_hold_oos: 29.83%
max_drawdown_consolidado: 6.12%
trades_total: 6
janelas_positivas: 0/3
```

Ou seja: com configuração pequena de AG, a estratégia não generalizou bem. Isso é uma descoberta boa, não ruim: o walk-forward está cumprindo o papel dele de separar "parece bom no treino" de "funciona no futuro".

Prompt 7: Vamos rodar com população e gerações maiores, por exemplo --population 40 --generations 25, e depois adicionar mais genes/indicadores além de SMA e RSI.

Rodamos o walk-forward maior em `GC=F` com `--population 40 --generations 25` e já adicionei novos genes/indicadores ao AG: `EMA`, `MACD`, `ATR` e os genes `min_entry_signals` / `min_exit_signals`, que fazem a estratégia comprar/vender quando uma quantidade mínima de sinais concorda.

Resultado novo fora da amostra, de `2023-01-03` até `2024-07-05`:

```text
capital_final = 11150.17
retorno_total = 11.50%
retorno_anual = 7.53%
buy_and_hold_oos = 29.83%
max_drawdown = 6.36%
trades_total = 43
janelas_positivas = 2/3
```

Comparando com a rodada anterior só com SMA/RSI em população 40 e 25 gerações: antes deu `-2.86%`, agora deu `+11.50%`. Melhorou bastante, mas ainda perdeu para comprar e segurar o ouro no mesmo período, que deu `+29.83%`.

Também corrigi um ponto importante no `walk_forward.py`: agora o período de teste usa o treino como "aquecimento" dos indicadores. Isso evita que médias longas, MACD e ATR fiquem sem histórico no começo da janela fora da amostra.

Prompt 8: Gostaria que fosse feita uma interface web para visualização da "execução" desse script. Mostrando o valor real, com os testes sendo feitos pelo AG.

Prompt 9: Vamos reduzir o overfitting. O AG pode "decorar o passado". Para combater isso: usar walk-forward separar treino/teste corretamente limitar genes absurdos penalizar estratégias com trades demais testar em vários períodos de mercado

Feito. Implementei uma camada anti-overfitting no AG e na interface web.

O que mudou:

1. O AG agora separa o próprio treino em duas partes: `otimização` e `validação interna`.
2. A estratégia não é mais escolhida só porque foi bem no treino inicial. Agora ela também precisa ir bem na validação.
3. Adicionei penalidade quando existe diferença grande entre: `resultado no treino` e `resultado na validação`.
4. Adicionei controle de quantidade de trades: `min_trades` e `max_trades`.
5. A interface web agora tem novos campos: `Validacao`, `Peso valid.`, `Penal. overfit`, `Min trades`, `Max trades`.
6. O walk-forward continua funcionando, mas agora cada janela também usa essa validação interna antes de testar fora da amostra.